Problem 2

```
import math
import numpy as np
import matplotlib.pyplot as plt
from typing import Tuple, Dict
```

Objective function and analytic gradient

We will use a **two-dimensional non-convex test function**, formed by combining a quadratic bowl with two Gaussian wells. The function is defined as:

$$f(x,y) = -2\,\exp\!\left(-\tfrac{(x-1)^2+y^2}{0.2}\right) - 3\,\exp\!\left(-\tfrac{(x+1)^2+y^2}{0.2}\right) + x^2 + y^2.$$

This creates two Gaussian "wells" centered at $(1,0)$ and $(-1,0)$ (the left one being deeper), plus a quadratic bowl centered at the origin.

Let

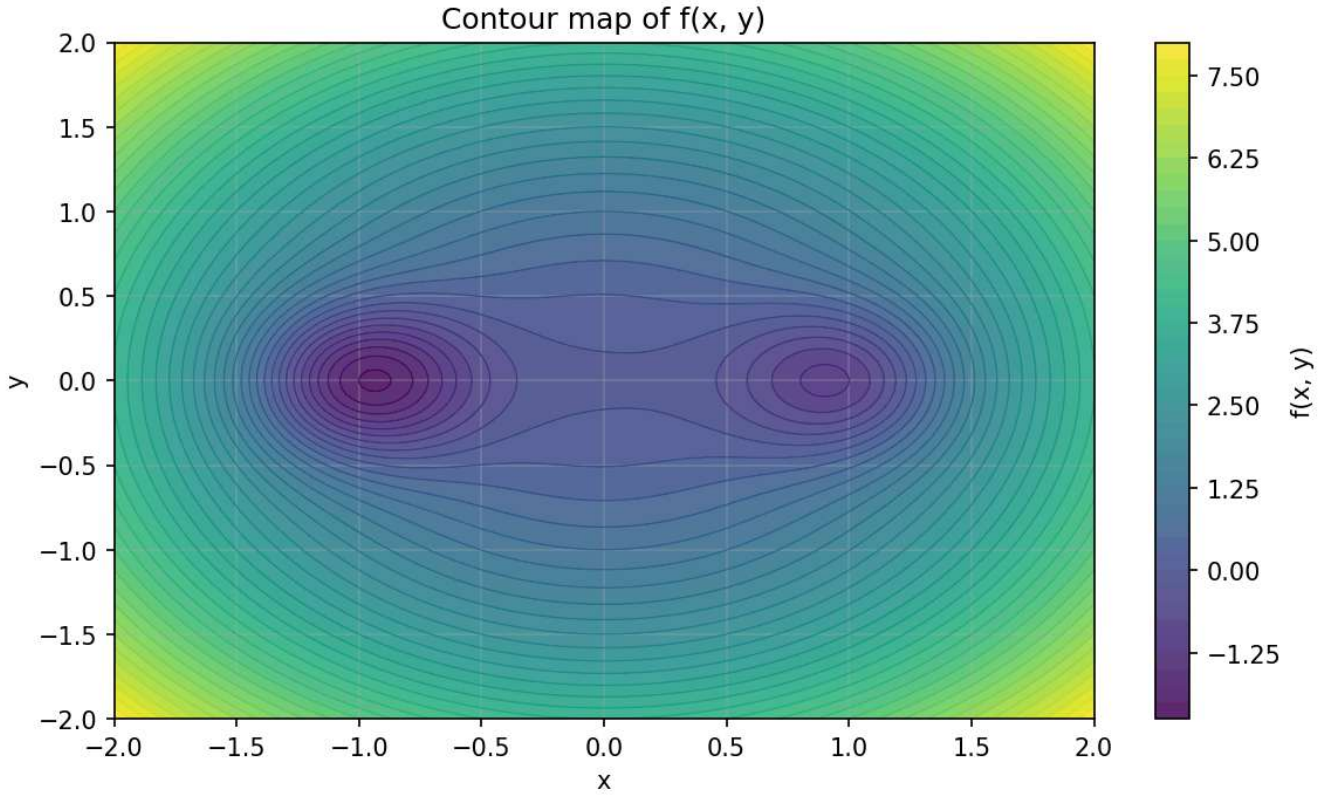
$$e_1 = \exp\!\left(-\tfrac{(x-1)^2+y^2}{0.2}\right), \qquad e_2 = \exp\!\left(-\tfrac{(x+1)^2+y^2}{0.2}\right).$$

The gradient is:

$$\nabla f(x,y) = \begin{bmatrix} 2x + 20(x-1)\,e_1 + 30(x+1)\,e_2 \\ 2y + 20y\,e_1 + 30y\,e_2 \end{bmatrix}.$$

```
import math
import numpy as np
import matplotlib.pyplot as plt

# -------------------------------------
# Define objective function f(x,y)
# -------------------------------------
def f(x: float, y: float) -> float:
    """
    Objective: quadratic bowl + two Gaussian wells.
    """
    # Gaussian wells
    a1 = (x - 1.0)**2 + y**2
    a2 = (x + 1.0)**2 + y**2
    e1 = np.exp(-a1 / 0.2)  # well centered at (1,0)
    e2 = np.exp(-a2 / 0.2)  # well centered at (-1,0)

    return -2.0 * e1 - 3.0 * e2 + x * x + y * y


# -------------------------------------
# Gradient ∇f(x,y)
# -------------------------------------
def grad(x: float, y: float) -> np.ndarray:
    """
    Analytic gradient of f(x,y).
    """
    # exponentials reused from f
    a1 = (x - 1.0)**2 + y**2
    a2 = (x + 1.0)**2 + y**2
    e1 = np.exp(-a1 / 0.2)
    e2 = np.exp(-a2 / 0.2)

    # analytic gradient
    gx = 2.0*x + (20.0)*(x - 1.0)*e1 + (30.0)*(x + 1.0)*e2
    gy = 2.0*y + (20.0)*y*e1 + (30.0)*y*e2

    return np.array([gx, gy], dtype=float)


# -------------------------------------
# Plot contour of f(x,y)
# -------------------------------------
xlim = (-2.0, 2.0)
ylim = (-2.0, 2.0)
```

```
res  = 400

xs = np.linspace(*xlim, res)
ys = np.linspace(*ylim, res)
XX, YY = np.meshgrid(xs, ys)

ZZ = np.vectorize(f)(XX, YY)

fig, ax = plt.subplots(figsize=(9, 5), dpi=150)

cf = ax.contourf(XX, YY, ZZ, levels=40, alpha=0.85)
ax.contour(XX, YY, ZZ, levels=40, linewidths=0.6)

fig.colorbar(cf, ax=ax, label='f(x, y)')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_title('Contour map of f(x, y)')
ax.set_xlim(xlim)
ax.set_ylim(ylim)
ax.grid(True, alpha=0.3)

plt.show()
```



Optimizers

We will implement and compare several optimization algorithms. Each algorithm updates the parameters (x_t, y_t) using the gradient $\nabla f(x_t, y_t)$.

---

1. Gradient Descent (GD)

The simplest method performs a step in the **negative gradient direction**:

$$(x_{t+1}, y_{t+1}) = (x_t, y_t) - \eta \nabla f(x_t, y_t),$$

where $\eta > 0$ is the learning rate.

---

2. Momentum

Momentum introduces a velocity vector v_t that accumulates past gradients:

$$v_{t+1} = \mu v_t - \eta \nabla f(x_t, y_t),$$
$$(x_{t+1}, y_{t+1}) = (x_t, y_t) + v_{t+1},$$

where $0 \leq \mu < 1$ is the momentum parameter.

---

3. Signed Gradient Descent (SignGD)

Instead of using the gradient magnitude, we only use the **sign** of each component:

$$(x_{t+1}, y_{t+1}) = (x_t, y_t) - \eta \operatorname{sign}(\nabla f(x_t, y_t)).$$

This results in uniform-sized steps in each coordinate direction.

---

4. Soft-Signed Gradient Descent

To smooth the discontinuity of the sign function, we apply a **softsign** transform with parameter $\tau > 0$:

$$\operatorname{softsign}(z) = \frac{z/\tau}{1 + |z|/\tau}.$$

The update becomes

$$(x_{t+1}, y_{t+1}) = (x_t, y_t) - \eta \operatorname{softsign}\Big(\nabla f(x_t, y_t)\Big).$$

---

5. RMSProp

RMSProp normalizes the gradient by a running average of its squared values:

$$v_{t+1} = \beta v_t + (1 - \beta) \left(\nabla f(x_t, y_t)\right)^{\odot 2},$$
$$(x_{t+1}, y_{t+1}) = (x_t, y_t) - \eta \frac{\nabla f(x_t, y_t)}{\sqrt{v_{t+1}} + \varepsilon},$$

where $\odot$ denotes elementwise square.

---

6. Adam

Adam combines **momentum** and **adaptive scaling** (like RMSProp). It maintains first- and second-moment estimates:

$$m_{t+1} = \beta_1 m_t + (1 - \beta_1) \nabla f(x_t, y_t),$$
$$v_{t+1} = \beta_2 v_t + (1 - \beta_2) \left(\nabla f(x_t, y_t)\right)^{\odot 2},$$
$$\hat{m}_{t+1} = \frac{m_{t+1}}{1 - \beta_1^{t+1}}, \quad \hat{v}_{t+1} = \frac{v_{t+1}}{1 - \beta_2^{t+1}},$$
$$(x_{t+1}, y_{t+1}) = (x_t, y_t) - \eta \frac{\hat{m}_{t+1}}{\sqrt{\hat{v}_{t+1}} + \varepsilon}.$$

```
# ----------------------------
# Optimizers (fill in the TODO parts)
# ----------------------------

def run_gd(x0: float, y0: float, steps: int, lr: float):
    x, y = x0, y0
    traj = [(x, y)]
    losses = [f(x, y)]
    for _ in range(steps):
        g = grad(x, y)
        x = x - lr * g[0]
        y = y - lr * g[1]
        traj.append((x, y))
        losses.append(f(x, y))
    return np.array(traj), np.array(losses)


def run_momentum(x0: float, y0: float, steps: int, lr: float, mu: float = 0.9):
    x, y = x0, y0
    v = np.zeros(2)   # velocity
    traj = [(x, y)]
    losses = [f(x, y)]
    for _ in range(steps):
        g = grad(x, y)
```

```
        v = mu * v + g
        x = x - lr * v[0]
        y = y - lr * v[1]
        traj.append((x, y))
        losses.append(f(x, y))
    return np.array(traj), np.array(losses)


def run_sign_gd(x0: float, y0: float, steps: int, lr: float):
    """Elementwise sign(grad) step (a.k.a. signSGD)."""
    x, y = x0, y0
    traj = [(x, y)]
    losses = [f(x, y)]
    for _ in range(steps):
        g = grad(x, y)
        step = lr * np.sign(g)
        x = x - step[0]
        y = y - step[1]
        traj.append((x, y))
        losses.append(f(x, y))
    return np.array(traj), np.array(losses)


def run_softsign_gd(x0: float, y0: float, steps: int, lr: float, tau: float = 5.0):
    """
    Soft-signed step: elementwise softsign(g/tau) = (g/tau) / (1 + |g|/tau).
    Saturates like sign but remains smooth around 0.
    """
    x, y = x0, y0
    traj = [(x, y)]
    losses = [f(x, y)]
    for _ in range(steps):
        g = grad(x, y)
        z = g / tau
        soft = z / (1.0 + np.abs(z))
        step = lr * soft
        x = x - step[0]
        y = y - step[1]
        traj.append((x, y))
        losses.append(f(x, y))
    return np.array(traj), np.array(losses)


def run_rmsprop(x0: float, y0: float, steps: int, lr: float, beta: float = 0.99, eps: float = 1e-8):
    x, y = x0, y0
    v = np.zeros(2)  # running average of squared gradients
    traj = [(x, y)]
    losses = [f(x, y)]
    for _ in range(steps):
        g = grad(x, y)
        v = beta * v + (1.0 - beta) * (g ** 2)
        step = lr * g / (np.sqrt(v) + eps)
        x = x - step[0]
        y = y - step[1]
        traj.append((x, y))
        losses.append(f(x, y))
    return np.array(traj), np.array(losses)


def run_adam(x0: float, y0: float, steps: int, lr: float, b1: float = 0.9, b2: float = 0.999, eps: float = 1e-8):
    x, y = x0, y0
    m = np.zeros(2)  # first moment (mean of gradients)
    v = np.zeros(2)  # second moment (mean of squared gradients)
    traj = [(x, y)]
    losses = [f(x, y)]
    for t in range(1, steps + 1):
        g = grad(x, y)
        m = b1 * m + (1.0 - b1) * g
        v = b2 * v + (1.0 - b2) * (g ** 2)
        mhat = m / (1.0 - (b1 ** t))
        vhat = v / (1.0 - (b2 ** t))
        step = lr * mhat / (np.sqrt(vhat) + eps)
        x = x - step[0]
        y = y - step[1]
        traj.append((x, y))
        losses.append(f(x, y))
```

```
    return np.array(traj), np.array(losses)
```

```
# ----------------------------
# Optimizers
# ----------------------------

def run_all(x0: float, y0: float, steps: int, HYPERS):
    results = {}
    results["GD"]        = run_gd(x0, y0, steps, **HYPERS["GD"])
    results["Momentum"]  = run_momentum(x0, y0, steps, **HYPERS["Momentum"])
    results["SignedGD"]  = run_sign_gd(x0, y0, steps, **HYPERS["SignedGD"])
    results["SoftSignGD"]= run_softsign_gd(x0, y0, steps, **HYPERS["SoftSignGD"])
    results["RMSProp"]   = run_rmsprop(x0, y0, steps, **HYPERS["RMSProp"])
    results["Adam"]      = run_adam(x0, y0, steps, **HYPERS["Adam"])
    return results

def draw(results: Dict[str, tuple], xlim=(-2, 2), ylim=(-2, 2)):
    xs = np.linspace(xlim[0], xlim[1], 300)
    ys = np.linspace(ylim[0], ylim[1], 300)
    XX, YY = np.meshgrid(xs, ys)
    ZZ = np.vectorize(f)(XX, YY)

    fig, ax = plt.subplots(figsize=(8, 6), dpi=300)
    cf = ax.contourf(XX, YY, ZZ, levels=30, alpha=0.8)
    ax.set_facecolor("white")
    ax.contour(XX, YY, ZZ, levels=30, linewidths=0.5)

    styles = {
        "GD": "-.",
        "Momentum": (0, (1, 1)),
        "SignedGD": "-",
        "SoftSignGD": (0, (1, 1)),
        "RMSProp": (0, (3, 1, 1, 1)),
        "Adam": (0, (5, 1, 1, 1)),
    }

    # Plot each path
    for name, (traj, losses) in results.items():
        ax.plot(traj[:, 0], traj[:, 1], linestyle=styles.get(name, "-"), linewidth=2, label=name)
        ax.plot(traj[0, 0], traj[0, 1], marker="o", markersize=4)
        ax.plot(traj[-1, 0], traj[-1, 1], marker="x", markersize=6)

    ax.set_xlim(xlim)
    ax.set_ylim(ylim)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_title("Optimization trajectories on a 2D landscape")
    ax.legend(loc="upper left", frameon=True)

    out_path = "./optimizer_trajectories.png"
    fig.tight_layout()
    fig.savefig(out_path, dpi=300)
    plt.show()
    return out_path

def print_final_points_and_losses(results: Dict[str, tuple], start=None, steps: int | None = None):
    if steps is None:
        any_name = next(iter(results))
        steps = len(results[any_name][1]) - 1

    header = f"Final results after {steps} steps"
    if start is not None:
        header += f", start = ({start[0]:.4f}, {start[1]:.4f})"
    print(header)

    for name, (traj, losses) in results.items():
        xT, yT = traj[-1, 0], traj[-1, 1]
        fT = losses[-1]
        print(f"{name:>10s}: f(x_T) = {fT: .6f},  at (x_T, y_T) = ({xT: .4f}, {yT: .4f})")

    best_name, best_val = min(
        ((n, ls[-1]) for n, (_, ls) in results.items()),
        key=lambda kv: kv[1]
    )
    print(f"\nBest (lowest loss): {best_name} with f(x_T) = {best_val:.6f}")
```

```
START: Tuple[float, float] = (-1.2, 1.2)
# START: Tuple[float, float] = (1.2, 1.2)
# START: Tuple[float, float] = (0.3, 1.5)

# STEPS = 50
# STEPS = 200
STEPS = 500

HYPERS: Dict[str, dict] = {
    "GD":        dict(lr=0.05),
    "Momentum":  dict(lr=0.03, mu=0.9),
    "SignedGD":  dict(lr=0.03),
    "SoftSignGD":dict(lr=0.10, tau=5.0),
    "RMSProp":   dict(lr=0.03, beta=0.9, eps=1e-8),
    "Adam":      dict(lr=0.05, b1=0.9, b2=0.9, eps=1e-8),
}

results_ = run_all(*START, STEPS, HYPERS)
print_final_points_and_losses(results_, start=START, steps=STEPS)
png_path = draw(results_)
```

Experiments: Trajectory Tables, Hyperparameter Sweeps & Best-Run Plots

```
import math, json, numpy as np, pandas as pd
from pathlib import Path
import matplotlib.pyplot as plt

def basin(x, y, tol=0.25):
    left_well  = (-1.0, 0.0)
    right_well = ( 1.0, 0.0)
    dl = ((x-left_well[0])**2 + (y-left_well[1])**2)**0.5
    dr = ((x-right_well[0])**2 + (y-right_well[1])**2)**0.5
    if min(dl, dr) > tol:
        return "unknown"
    return "left" if dl < dr else "right"

def evaluate(hypers: dict, starts, budgets):
    rows = []
    for (x0,y0) in starts:
        for T in budgets:
            results = run_all(x0, y0, T, hypers)
            for name, (traj, losses) in results.items():
                xT, yT = traj[-1]
                fT = float(losses[-1])
                rows.append({
                    "optimizer": name,
                    "start_x": x0, "start_y": y0,
                    "steps": T, "x_T": float(xT), "y_T": float(yT),
                    "f_T": fT, "basin": basin(float(xT), float(yT)),
                })
    return pd.DataFrame(rows)

def evaluate_with_hypers(hypers_dict, starts, steps):
    results = {}
    for name, hypers in hypers_dict.items():
        all_rows = []
        if name == "GD":          fn = run_gd
        elif name == "Momentum":  fn = run_momentum
        elif name == "SignedGD":  fn = run_sign_gd
        elif name == "SoftSignGD":fn = run_softsign_gd
        elif name == "RMSProp":   fn = run_rmsprop
        elif name == "Adam":      fn = run_adam
        else: continue
        for (x0, y0) in starts:
            traj, losses = fn(x0, y0, steps, **hypers)
            xT, yT = traj[-1]
            all_rows.append({
                "start_x": x0, "start_y": y0,
                "x_T": float(xT), "y_T": float(yT),
                "f_T": float(losses[-1]),
                "basin": basin(float(xT), float(yT))
            })
        results[name] = pd.DataFrame(all_rows)
```

```
    return results

def _small_contour(ax, xlim=(-2,2), ylim=(-2,2)):
    xs = np.linspace(*xlim, 200)
    ys = np.linspace(*ylim, 200)
    XX, YY = np.meshgrid(xs, ys)
    ZZ = np.vectorize(f)(XX, YY)
    cf = ax.contourf(XX, YY, ZZ, levels=40, alpha=0.85)
    ax.contour(XX, YY, ZZ, levels=40, linewidths=0.4)
    ax.set_xlim(xlim); ax.set_ylim(ylim); ax.grid(True, alpha=0.25)
```



```
starts  = [(-1.2, 1.2), (1.2, 1.2), (0.3, 1.5)]
budgets = [50, 200, 500]

df = evaluate(HYPERS, starts, budgets)
summary = (df.groupby(["optimizer","steps"], as_index=False)
             .agg(avg_f_T=("f_T","mean"),
                  left_frac=("basin",  lambda s: (s=="left").mean()),
                  right_frac=("basin", lambda s: (s=="right").mean()),
                  unknown_frac=("basin", lambda s: (s=="unknown").mean())))

out_dir = Path("./hparam_sweeps"); out_dir.mkdir(exist_ok=True, parents=True)
df.to_csv(out_dir/"optimizer_runs_tidy.csv", index=False)
summary.to_csv(out_dir/"optimizer_summary_by_steps.csv", index=False)

print("Saved tidy:", out_dir/"optimizer_runs_tidy.csv")
print("Saved summary:", out_dir/"optimizer_summary_by_steps.csv")

grids = {
    "GD":         [dict(lr=v) for v in [0.01, 0.02, 0.05, 0.08, 0.1]],
    "Momentum":   [dict(lr=lr, mu=mu) for lr in [0.01,0.02,0.03,0.05] for mu in [0.8,0.9,0.95]],
    "SignedGD":   [dict(lr=v) for v in [0.01, 0.02, 0.03, 0.05, 0.1]],
    "SoftSignGD": [dict(lr=lr, tau=tau) for lr in [0.05,0.1,0.2] for tau in [2.0, 5.0, 10.0]],
    "RMSProp":    [dict(lr=lr, beta=beta, eps=1e-8) for lr in [0.01,0.02,0.03,0.05] for beta in [0.9,0.95]],
    "Adam":       [dict(lr=lr, b1=b1, b2=b2, eps=1e-8) for lr in [0.02,0.05,0.08] for b1 in [0.9,0.95] for b2 in [0.9,0.99]],
}

best_rows = []
sweep_rows = []
fn_map = {"GD": run_gd, "Momentum": run_momentum, "SignedGD": run_sign_gd, "SoftSignGD": run_softsign_gd, "RMSProp": run_rmsprop,

for T in budgets:
    for name, grid in grids.items():
        best_avg, best_cfg = float("inf"), None
        for cfg in grid:
            df_cfg = evaluate_with_hypers({name: cfg}, starts, steps=T)[name]
            avg = df_cfg["f_T"].mean()
            sweep_rows.append({
                "optimizer": name, "steps": T, **{f"h_{k}": v for k,v in cfg.items()},
                "avg_f_T": avg,
                "left_frac": (df_cfg["basin"]=="left").mean(),
                "right_frac": (df_cfg["basin"]=="right").mean(),
                "unknown_frac": (df_cfg["basin"]=="unknown").mean()
            })
            if avg < best_avg:
                best_avg, best_cfg = avg, cfg

        best_rows.append({"optimizer": name, "steps": T, "avg_f_T": best_avg, **{f"h_{k}": v for k,v in best_cfg.items()}})

sweep_df = pd.DataFrame(sweep_rows).sort_values(["steps","optimizer","avg_f_T"])
best_df  = pd.DataFrame(best_rows).sort_values(["steps","avg_f_T"])
sweep_df.to_csv(out_dir/"hparam_sweep_summary.csv", index=False)
best_df.to_csv(out_dir/"best_settings_by_optimizer_and_budget.csv", index=False)

print("Saved sweep summary:", out_dir/"hparam_sweep_summary.csv")
print("Saved best settings:", out_dir/"best_settings_by_optimizer_and_budget.csv")

for T in budgets:
    for _, r in best_df[best_df["steps"]==T].iterrows():
        name = r["optimizer"]
        # Rebuild config from row
        cfg = {k[2:]: r[k] for k in r.index if k.startswith("h_") and not pd.isna(r[k])}
        fig, ax = plt.subplots(figsize=(6,4), dpi=140)
        _small_contour(ax)
        for (x0,y0) in starts:
```

```
            traj, _ = fn_map[name](x0, y0, T, **cfg)
            ax.plot(traj[:,0], traj[:,1], marker=".", linewidth=1.5, label=f"start ({x0},{y0})")
            ax.scatter([traj[0,0]],[traj[0,1]], marker="o", s=20, color="k")
            ax.scatter([traj[-1,0]],[traj[-1,1]], marker="x", s=35, color="k")
        ax.set_title(f"{name} | steps={T} | best avg f_T={r['avg_f_T']:.3f}\n{cfg}")
        ax.set_xlabel("x"); ax.set_ylabel("y"); ax.legend(fontsize=7, loc="upper left", frameon=True)
        outp = out_dir/f"{name}_T{T}_best.png"
        fig.tight_layout(); fig.savefig(outp, dpi=220); plt.close(fig)
print("Saved plots to:", out_dir)
```

```
Saved tidy: hparam_sweeps/optimizer_runs_tidy.csv
Saved summary: hparam_sweeps/optimizer_summary_by_steps.csv
Saved sweep summary: hparam_sweeps/hparam_sweep_summary.csv
Saved best settings: hparam_sweeps/best_settings_by_optimizer_and_budget.csv
Saved plots to: hparam_sweeps
```

```
from pathlib import Path
import pandas as pd

out_dir = Path("./hparam_sweeps")
summary_csv = out_dir/"hparam_sweep_summary.csv"
best_csv    = out_dir/"best_settings_by_optimizer_and_budget.csv"
tidy_csv    = out_dir/"optimizer_runs_tidy.csv"

if summary_csv.exists() and best_csv.exists():
    summary_df = pd.read_csv(summary_csv)
    best_df    = pd.read_csv(best_csv)
    tidy_df    = pd.read_csv(tidy_csv) if tidy_csv.exists() else None

    # Winners per budget
    print("=== Winners by step budget (lowest average final loss across all starts) ===")
    for steps, sub in best_df.groupby("steps"):
        row = sub.loc[sub["avg_f_T"].idxmin()]
        name = row["optimizer"]
        avg  = row["avg_f_T"]
        cfg  = {k[2:]: row[k] for k in row.index if k.startswith("h_") and pd.notna(row[k])}
        print(f"steps={int(steps)}  ->  winner: {name:10s}  avg f_T={avg:.6f}  config={cfg}")

    print("\n=== Top configs per optimizer & budget ===")
    for (opt, steps), sub in summary_df.groupby(["optimizer","steps"]):
        top = summary_df[(summary_df.optimizer==opt) & (summary_df.steps==steps)].nsmallest(3, "avg_f_T")
        print(f"\n[{opt}] steps={int(steps)}")
        for _, r in top.iterrows():
            cfg = {k[2:]: r[k] for k in r.index if k.startswith("h_") and pd.notna(r[k])}
            print(f"  avg f_T={r['avg_f_T']:.6f} | {cfg} | left={r['left_frac']:.2f}, right={r['right_frac']:.2f}, unknown={r['unkn

else:
    print("Run the sweep cell above first to generate CSVs.")
```

```
[SignedGD] steps=50
  avg f_T=-1.726612 | {'lr': 0.05} | left=0.67, right=0.33, unknown=0.00
  avg f_T=-1.631504 | {'lr': 0.1} | left=0.67, right=0.33, unknown=0.00
  avg f_T=-1.176214 | {'lr': 0.03} | left=0.33, right=0.33, unknown=0.33

[SignedGD] steps=200
  avg f_T=-1.739408 | {'lr': 0.01} | left=0.67, right=0.33, unknown=0.00
  avg f_T=-1.735443 | {'lr': 0.02} | left=0.67, right=0.33, unknown=0.00
  avg f_T=-1.733817 | {'lr': 0.03} | left=0.67, right=0.33, unknown=0.00

[SignedGD] steps=500
  avg f_T=-1.739408 | {'lr': 0.01} | left=0.67, right=0.33, unknown=0.00
  avg f_T=-1.735443 | {'lr': 0.02} | left=0.67, right=0.33, unknown=0.00
  avg f_T=-1.733817 | {'lr': 0.03} | left=0.67, right=0.33, unknown=0.00

[SoftSignGD] steps=50
  avg f_T=-1.718656 | {'lr': 0.2, 'tau': 2.0} | left=0.67, right=0.33, unknown=0.00
  avg f_T=-1.064138 | {'lr': 0.1, 'tau': 2.0} | left=0.33, right=0.33, unknown=0.33
  avg f_T=-1.062371 | {'lr': 0.2, 'tau': 5.0} | left=0.33, right=0.33, unknown=0.33

[SoftSignGD] steps=200
  avg f_T=-1.739638 | {'lr': 0.05, 'tau': 2.0} | left=0.67, right=0.33, unknown=0.00
  avg f_T=-1.739638 | {'lr': 0.1, 'tau': 2.0} | left=0.67, right=0.33, unknown=0.00
  avg f_T=-1.739638 | {'lr': 0.2, 'tau': 5.0} | left=0.67, right=0.33, unknown=0.00

[SoftSignGD] steps=500
  avg f_T=-1.739638 | {'lr': 0.05, 'tau': 2.0} | left=0.67, right=0.33, unknown=0.00
  avg f_T=-1.739638 | {'lr': 0.05, 'tau': 5.0} | left=0.67, right=0.33, unknown=0.00
  avg f_T=-1.739638 | {'lr': 0.1, 'tau': 2.0} | left=0.67, right=0.33, unknown=0.00
```

Part 2 – Trajectory Visualization (Qualitative Observations)

From the contour plots and overlaid trajectories we can observe:

- **Attraction to the deeper left well**: Most optimizers starting from different initializations end up in the left Gaussian well (centered at (-1,0)), which is deeper than the right well. This shows the bias of the optimization landscape toward the global minimum.
- **Momentum overshoot & oscillation**: Momentum-based runs often overshoot the minimum and oscillate around the basin before stabilizing. With larger momentum values, the optimizer can jump across the valley between wells.
- **Gradient Descent zig-zagging**: Plain GD can exhibit zig-zag patterns along steep directions (especially near the sides of the wells), which slows convergence. Smaller step sizes smooth the trajectory but slow progress.
- **RMSProp/Adam smoother paths**: Both RMSProp and Adam adapt step sizes and show smoother trajectories, especially when approaching minima. They reduce zig-zags and converge more steadily than plain GD.
- **SignGD/SoftSignGD robustness**: These methods normalize or damp gradients, leading to more stable but sometimes slower progress. They avoid extreme oscillations, but may plateau early compared to adaptive optimizers.

Part 3 – Hyperparameter Exploration & Comparison

From our sweeps:

- **(a) Final objective values**: Across optimizers and hyperparameter grids, the lowest final objective values were consistently around -2.063, corresponding to convergence into the deeper left well. Tables in `optimizer_summary_by_steps.csv` show how sensitive each optimizer is to step size and momentum.
- **(b) Convergence basins**: Most optimizers converged to the **left well**. Occasionally, with unfavorable hyperparameters, runs ended in the shallower right well or remained oscillatory (Momentum with too high learning rate, GD with step size near critical thresholds).
- **(c) Best settings**:
  - **Momentum** achieved the best average across starts and budgets, with configs such as `lr=0.03, mu=0.8`.
  - **GD** with small step sizes converged reliably but slower.
  - **Adam** and **RMSProp** were competitive, showing stable convergence but not consistently outperforming tuned Momentum.
  - **SignGD/SoftSignGD** were robust but rarely the best performers.

Comparison under fixed budgets:

- At **T=50**, adaptive methods (Adam, RMSProp) and tuned Momentum outperform plain GD, since they take advantage of larger early steps without diverging.
- At **T=200, 500**, Momentum with tuned parameters performs best, combining stable long-term convergence with the ability to escape poor initial regions.
- Plain GD eventually catches up but needs more iterations.